

SKRIP TUTORIAL

Tutorial Python di Machine Learning

Oleh :

Lucia Dwi KRISNAWATI

Materi Tambahan

Machine Learning (TI6043)

Prodi Informatika

Fakultas Teknologi Informasi

UNIVERSITAS KRISTEN DUTA WACANA

Yogyakarta, 29 Oktober 2025

Daftar Isi

1	Pengantar	3
2	Model Klasifikasi	3
2.1	Membaca Berkas CSV	3
2.2	Naive Bayes	4
2.2.1	GNB	4
2.2.2	Multinomial Naive Bayes	5
2.3	k-Nearest Neighbor	6

Tutorial python untuk Machine Learning

1 Pengantar

Dengan asumsi bahwa kalian sudah belajar tahap prapemrosesan dan seleksi fitur, maka tutorial di sini langsung menuju ke model, khususnya model Pembelajaran Terbimbing (*supervised learning*).

2 Model Klasifikasi

Sebelum kita belajar model klasifikasi, alangkah baiknya belajar membaca berkas csv yang menyimpan fitur-fitur yang telah diekstraksi dari proses sebelumnya.

2.1 Membaca Berkas CSV

Ada dua cara untuk memanggil dan membaca data fitur yang terekstraksi dari sampel citra bunga Iris, yakni:

1. dengan cara mengimpornya dari pustaka sklearn:

```
from sklearn.datasets import load_iris

dataset = load_iris()
print(dataset)
```

Hasilnya, data akan dibaca sebagai list seperti yang ditunjukkan di Gambar 1.

2. Membaca berkas csv dengan pustaka *Pandas*. Jika membaca berkas csv dengan pustaka ini, maka perlu mendefinisikan atribut tiap fitur atau judul kolom dalam basis data kalian. Contohnya sbb:

```
import pandas # pustaka untuk memanipulasi dan analisis data
```

```

===== RESTART: D:/lectures/Gasal1920/SuLe/bahan/classi
{'data': array([[5.1, 3.5, 1.4, 0.2],
                [4.9, 3. , 1.4, 0.2],
                [4.7, 3.2, 1.3, 0.2],
                [4.6, 3.1, 1.5, 0.2],
                [5. , 3.6, 1.4, 0.2],
                [5.4, 3.9, 1.7, 0.4],
                [4.6, 3.4, 1.4, 0.3],
                [5. , 3.4, 1.5, 0.2],
                [4.4, 2.9, 1.4, 0.2],
                [4.9, 3.1, 1.5, 0.1],
                [5.4, 3.7, 1.5, 0.2],
                [4.8, 3.4, 1.6, 0.2],
                [4.8, 3. , 1.4, 0.1],
                [4.3, 3. , 1.1, 0.1],
                [5.8, 4. , 1.2, 0.2],
                [5.7, 4.4, 1.5, 0.4],
                [5.4, 3.9, 1.3, 0.4],
                [5.1, 3.5, 1.4, 0.3],
                [5.7, 3.8, 1.7, 0.3],
                [5.1, 3.8, 1.5, 0.3],
                [5.4, 3.4, 1.7, 0.2],

```

Gambar 1: Dataset Iris yang dipanggil dari pustaka sklearn. Hasil dibaca sebagai list

```

# membaca file csv dan menyimpannya dalam variabel
doc = "iris.csv"
names = ['sepal-length', 'sepal-width', 'petal-length',
        'petal-width', 'class']
dataset = pandas.read_csv(doc, names=names)
print(dataset)

```

Hasilnya dibaca sebagai matriks dengan format berkas csv juga. Lihat Gambar 2.

Untuk penjelasan tentang model klasifikasi Bayes Theorem, konsep dan landasan pemikirannya bisa dilihat di materi kuliah. Berikut ini contoh aplikasi beberapa varian model Bayes klasifikator di Python dengan menggunakan pustaka scikit-learn.

2.2 Naive Bayes

Ada beberapa varian Naive Bayes, dan contoh yang diberikan meliputi aplikasi Gaussian Naive Bayes (GNB) dan Multinomial Naives Bayes (MNB).

2.2.1 Gaussian Naive Bayes

Pengkodeannya dengan menggunakan sklearn adalah sebagai berikut:

```

from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import classification_report

```

```
Python 3.6.4 Shell
File Edit Shell Debug Options Window Help
Python 3.6.4 (v3.6.4:d48eceb, Dec 19 2017, 06:04:45) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: D:\lectures\Gasal1920\SuLe\bahan\classifiers.py =====
   sepal-length  sepal-width  ...  petal-width  class
0          5.1          3.5  ...          0.2  Iris-setosa
1          4.9          3.0  ...          0.2  Iris-setosa
2          4.7          3.2  ...          0.2  Iris-setosa
3          4.6          3.1  ...          0.2  Iris-setosa
4          5.0          3.6  ...          0.2  Iris-setosa
5          5.4          3.9  ...          0.4  Iris-setosa
6          4.6          3.4  ...          0.3  Iris-setosa
7          5.0          3.4  ...          0.2  Iris-setosa
8          4.4          2.9  ...          0.2  Iris-setosa
9          4.9          3.1  ...          0.1  Iris-setosa
10         5.4          3.7  ...          0.2  Iris-setosa
11         4.8          3.4  ...          0.2  Iris-setosa
12         4.8          3.0  ...          0.1  Iris-setosa
13         4.3          3.0  ...          0.1  Iris-setosa
14         5.8          4.0  ...          0.2  Iris-setosa
15         5.7          4.4  ...          0.4  Iris-setosa
16         5.4          3.9  ...          0.4  Iris-setosa
17         5.1          3.5  ...          0.3  Iris-setosa
```

Gambar 2: Dataset Iris yang dipanggil dengan pustaka pandas. Hasil dibaca sebagai matriks

```
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn import model_selection
import pandas as pd

model = GaussianNB()
name = "GNB"

X =dataset.iloc[1: , 0:4]
Y =dataset.iloc[1: , 4]
X_train, X_test, Y_train, Y_test = model_selection.train_test_split(X, Y,
    test_size=0.30, random_state=100)
model.fit(X_train,Y_train)
predicted = model.predict(X_test)
print('Klasifikasi model: ', name)
print('Accuracy score', round(accuracy_score(Y_test,
    predicted), 2))
print(classification_report(Y_test, predicted))
```

2.2.2 Multinomial Naive Bayes

Pengkodeannya dengan menggunakan sklearn adalah sebagai berikut:

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score

model = MultinomialNB()
name = "MNB"
model.fit(X_train,Y_train)
predicted = model.predict(X_test)
print('Accuracy score', round(accuracy_score(Y_test,
    predicted), 2))
print(classification_report(Y_test, predicted))
```

2.3 *k*-Nearest Neighbor

Sesuai namanya, *k*-Nearest Neighbor (KNN) adalah model yang mencari jarak tetangga yang terdekat. Ini dilakukan dengan mengkomputasi jarak antara data atau dokumen tes dengan setiap data latih kemudian dilakukan perankingan dengan jarak yang terdekat. *K* dalam *k*-NN merupakan variabel yang menentukan jumlah tetangga terdekat yang menentukan komputasinya. Jika *k* ditentukan sebanyak 5, maka 5 tetangga terdekat, yang berdasarkan ranking jarak terdekatnya berada di daftar 5 teratas, akan dipakai sebagai acuan kelasnya. Dari 5 tetangga ini, jumlah kelas atau kategori terbanyak akan dijadikan kelas dari data tes.

Contoh berikut ini akan mempermudah pemahaman tentang *k*-NN. Asumsikan kalau kita melakukan pengenalan wajah kucing saya dengan kucing tetangga. Jadi luaran pengenalan ini berbentuk biner yakni kucing saya dan kucing tetangga yang mencakup semua kucing selain kucing saya. Di data latih, ada banyak sampel gambar kucing saya dan berbagai gambar kucing-kucing tetangga. Ada 1 gambar kucing A yang dimasukkan ke sistem sebagai data tes. Saat model *k*-NN kita jalankan dan *k* ditetapkan *k*=5, maka model akan mengkomputasi sesuai fitur-fitur yang ditetapkan setiap jarak gambar kucing A dengan setiap gambar kucing yang ada di data latih. Jarak ini kemudian diurutkan dari nilai jarak terendah ke tinggi. Kemudian 5 nilai jarak terendah beserta data lengkapnya diambil oleh sistem dan dilihat labelnya. Asumsikan anotasi data di 5 data terdekat adalah sebagai berikut:

- Ranking 1 memiliki label ‘kucing tetangga’.
- Ranking 2 memiliki label ‘kucing saya’,
- Ranking 3 memiliki label ‘kucing tetangga’.
- Ranking 4 memiliki label ‘kucing tetangga’.
- Ranking 2 memiliki label ‘kucing saya’,

K-NN selanjutnya melakukan voting bagi kelas yang ada, dan dari hasil di atas, maka kategori *kucing tetangga* memiliki nilai voting 3 sedangkan *kucing saya* memiliki nilai voting 2. Maka gambar kucing A diberi label *kucing tetangga* karena nilai votingnya lebih tinggi daripada kucing saya.

Pengkodeannya menggunakan SKLearn bisa dilakukan dengan memanggil kelas *neighbors* sebagai berikut:

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score

model = KNeighborsClassifier(n_neighbors=7)
name = "KNN"
model.fit(X_train,Y_train)
predicted = model.predict(X_test)
print('Accuracy score', round(accuracy_score(Y_test,
    predicted), 2))
print(classification_report(Y_test, predicted))
```

Dari snippet kode program di atas bahwa yang membedakan model k-NN dengan Naive Bayes adalah baris yang mengimport *KNeighborsClassifier* dan *assignment* model k-NN ke dalam objek **model**. Juga nilai dari variabel ‘name’ diganti dan disesuaikan dengan odel atau klasifikatornya. Selain itu, kodenya sama. Penjelasan singkatnya sebagai berikut:

- Klasifikasi k-NN hanya memiliki 1 parameter saja, yakni jumlah tetangga terdekat atau k. Dalam contoh ini k= 7
- Untuk pelatihan, maka dipanggilah fungsi atau metode *fit()* yang memiliki 2 parameter, yakni sampel fitur data latih dan fitur data tes.

- Untuk melakukan prediksi, digunakan fungsi `predict()` yang hanya membutuhkan 1 parameter saja, yakni sampel fitur data tes.